

“ FloorBiz ”

A PROJECT REPORT

Submitted by

NAME OF THE CANDIDATE(S)

1. Aditya Anil Tippanna
2. Samyak Nitin Wadmare
3. Vishwajeet Rahul Karkar
4. Shital Manoj Patil

**B.Tech
IN
COMPUTER SCIENCE AND ENGINEERING**



**KOLHAPUR INSTITUTE OF TECHNOLOGY'S
COLLEGE OF ENGINEERING (AUTONOMOUS),
KOLHAPUR**

April 2026

**KOLHAPUR INSTITUTE OF TECHNOLOGY'S
COLLEGE OF ENGINEERING (AUTONOMOUS),
KOLHAPUR**

CERTIFICATE

This is to certify that the Project report entitled, “ **FloorBiz** ” submitted by “**Aditya Tippanna(TY B42)** **Samyak Wadmare (TY B44), Vishwajeet Karkar (TY B74), Shital Patil (TY B77)**”, in partial fulfillment for the award of the degree of “ **BTech** ” in “ *Computer Science and Engineering* ” at KIT's College of Engineering, Kolhapur, Maharashtra, INDIA, is a record of their own work carried out under my / our supervision and guidance.

Dr. Lingaraj Hadimani
HOD
Dept. of CSE

Mr. Avinash Powar
Asst. Professor
Dept. of CSE

**KOLHAPUR INSTITUTE OF TECHNOLOGY'S
COLLEGE OF ENGINEERING (AUTONOMOUS),
KOLHAPUR**

DECLARATION

I hereby declare that the Project entitled, “ ***FloorBiz*** ” submitted to KIT’s College of Engineering, Kolhapur, Maharashtra, INDIA in the partial fulfillment of the award of the Degree of “***BTECH***” in “ ***Computer Science and Engineering*** ” is a bonafide work carried out by us. The material contained in this Seminar/ Project has not been submitted to any University or Institution for the award of any degree.

Place: Kolhapur.
Date: April 2026

KOLHAPUR INSTITUTE OF TECHNOLOGY'S COLLEGE OF ENGINEERING (AUTONOMOUS), KOLHAPUR

Acknowledgements

We would like to express our sincere gratitude to Mr. Avinash Powar for his invaluable guidance and support throughout this project. his insights, encouragement, and expertise have been instrumental in shaping our work, and we deeply appreciate his mentorship.

We also extend our thanks to everyone who contributed directly or indirectly to the successful completion of this project.

Place: Kolhapur
Date: Apr 2026

TABLE OF CONTENTS

Chapter No.	Title	Page No.
Cover	Cover Page	—
Certificate	Certificate	1
Declaration	Declaration	2
Acknowledgements	Acknowledgements	3
Contents	Table of Contents	4
1	Introduction	5
2	Project Overview & Architecture	6
3	Quick Start Guide	8
4	Viewer Features	10
5	Material System	12
6	API Reference	14
7	Known Limitations	15
8	Troubleshooting	16

1. Introduction

1.1 Overview

The Floor Plan 3D Visualizer is a full-stack application that allows users to upload a floor plan image (PNG, JPG, PDF) or DXF file and receive an interactive, textured 3D model in return. The system is designed to bridge the gap between flat architectural drawings and immersive three-dimensional representations, making it accessible to students, architects, and hobbyists alike.

The application is built on a two-component stack:

- FastAPI (Python 3.10+) — a high-performance backend that handles file ingestion, image processing, and 3D geometry construction.
- Three.js (browser, no build step) — a self-contained frontend viewer that renders the resulting 3D model with real-time controls, multiple render modes, and a virtual tour feature.

The project is hosted on GitHub at:

<https://github.com/Shital-P276/6-MP/tree/v2.2>

1.2 Purpose of the report

This report provides a comprehensive account of the Floor Plan 3D Visualizer project — its architecture, features, setup process, and known limitations. It serves as both a technical reference for developers and a formal academic submission documenting the team's design decisions and implementation choices.

1.3 Background and motivation

Architectural floor plans are traditionally viewed as 2D drawings, which can make spatial understanding difficult for non-specialists. The motivation

behind this project was to create a lightweight, browser-based tool that automatically converts a scanned or digital floor plan into a navigable 3D model — without requiring expensive software like AutoCAD or Revit.

Key pain points this project addresses:

- Manual 3D modeling is time-consuming and requires specialist skills.
- Existing online tools often require proprietary file formats or subscriptions.
- Students and small offices need a zero-cost, zero-install solution.

1.4 Objectives

The primary objectives of the project are:

- Automated wall, door, and window detection from raster floor plan images.
- Accurate 3D geometry construction exported as Three.js-compatible JSON.
- An interactive browser viewer with orbit, pan, zoom, and multiple view modes.
- A virtual tour feature for first-person walkthrough of detected rooms.
- A material system with procedurally generated textures requiring no external assets.
- A clean REST API enabling future integration with other tools.

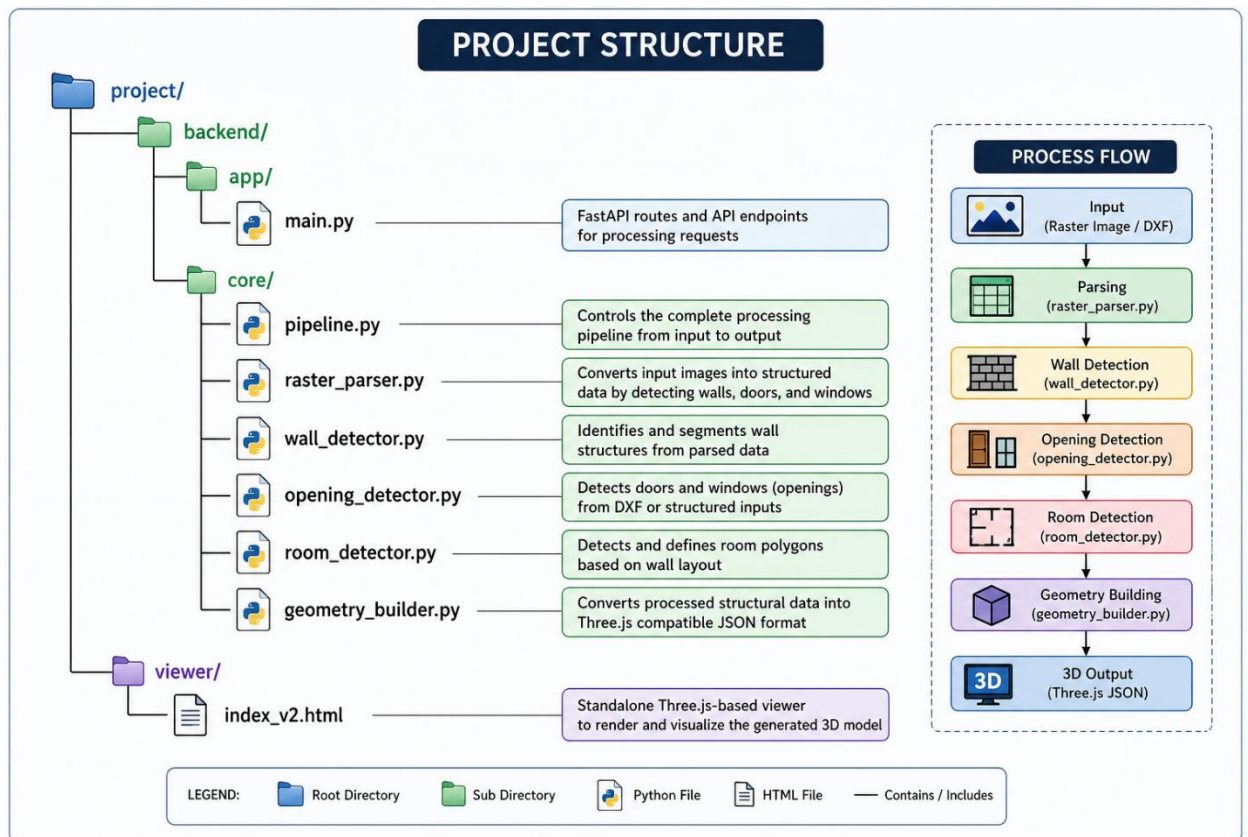
2. Project Overview & Architecture

2.1 Technology Stack

Layer	Technology	Purpose
Backend	FastAPI (Python 3.10+)	REST API, file handling, image processing
Image Processing	OpenCV / raster_parser.py	Wall/door/window detection from images
Geometry	geometry_builder.py	Converts detected walls to Three.js JSON
Frontend	Three.js (CDN, no build)	Interactive 3D rendering in the browser
Server	Uvicorn (ASGI)	High-performance async HTTP server
Static Serving	Python http.server	Lightweight local file server for viewer

2.2 Project Structure

The repository is organized into two top-level directories:



2.3 Data Flow

The processing pipeline follows these steps in order:

1. User uploads a floor plan (PNG/JPG/PDF/DXF) via the browser viewer.
2. FastAPI receives the file and assigns a unique `job_id`.
3. The `pipeline.py` orchestrator calls `raster_parser.py` to detect walls, doors, and windows.
4. `room_detector.py` identifies closed room polygons from the detected walls.
5. `geometry_builder.py` converts all geometry into Three.js-compatible JSON.
6. The viewer fetches the JSON and renders the 3D model in the browser.

3. Quick Start Guide

3.1 Prerequisites

- Python 3.10 or higher
- pip (Python package manager)
- A modern web browser (Chrome, Firefox, Edge)
- Node.js is NOT required — the viewer uses Three.js via CDN

3.2 Step 1 — Start the Backend

Navigate to the backend directory and install dependencies, then start the Uvicorn server:

```
cd backend pip install -r requirements.txt uvicorn  
app.main:app --reload --port 8000
```

On success you will see: Uvicorn running on `http://127.0.0.1:8000`

3.3 Step 2 — Serve the Viewer

Important: Do NOT open `index_v2.html` directly as a file. Browsers block `fetch()` calls to localhost from `file://` URLs. You must serve it via HTTP.

```
# From the viewer/ folder: python -m http.server 3000
```

Then open `http://localhost:3000/index_v2.html` in your browser. The green dot in the top-right header should show API CONNECTED.

3.4 Step 3 — Upload a Floor Plan

Drop a PNG, JPG, PDF, or DXF floor plan into the upload zone (or click to browse), then press PROCESS FLOOR PLAN. The viewer will fetch the processed 3D model and render it automatically.

Note: Always upload the original floor plan PNG, not a screenshot of the 3D viewer, for best detection results.

4. Viewer Features

4.1 Render Modes

Blueprint Mode (Default)

Dark navy walls with cyan wireframes rendered over a dark grid background — the classic technical drawing aesthetic. This is the default mode on startup.

Realistic Mode

Neutral white lighting with procedurally generated textures applied to walls and floors. When active, an amber REALISTIC badge appears on the canvas. Toggle between modes using the right panel.

4.2 Camera Controls

Action	Control
Orbit (rotate)	Left mouse drag
Zoom	Scroll wheel
Pan	Right mouse drag
Perspective view	Key 1 or PERSP button
Top view	Key 2 or TOP button
Front view	Key 3 or FRONT button
Side view	Key 4 or SIDE button
Toggle solid geometry	Key S or SOLID button
Toggle wireframe	Key W or WIRE button
Toggle floor	Key F or FLOOR button
Toggle doors/windows	DOORS / WIN buttons
Process floor plan	Spacebar

4.3 Virtual Tour

After a floor plan is processed and rooms are detected, the START

TOUR button activates a first-person walkthrough of every room. The button is disabled until a model with detected rooms is loaded.

How it Works

- Hotspots — a pulsing purple dot appears on the floor at each room's centroid. Hovering shows the room name; clicking flies the camera there.
- Camera fly — smooth cubic ease-in-out animation from the current viewpoint to eye height (1.6 m) inside the target room. Orbit controls remain active after landing.
- HUD nav bar — appears at the bottom with previous/next arrows, room name, index counter (e.g. ROOM 2 OF 4), area display, progress pips, and an autoplay button that advances every 4 seconds.
- Minimap — bottom-right panel showing all rooms as numbered dots. The active room is highlighted and a line indicates the camera look direction.
- Crosshair — subtle centre-screen indicator active during tour mode.

Tour Controls

Action	Control
Start / exit tour	Key T or sidebar button
Fly to a room	Click a floor hotspot
Previous / next room	← / → HUD arrows or pip dots
Toggle autoplay	▶ / button in HUD

5. Material System

The right panel can be opened or closed using the ◀ ▶ button on its left edge. All textures in the system are procedurally generated — no external image files are required.

5.1 Wall Finishes

Texture	Description
PLASTER	Warm grey matte with surface variation
BRICK	Red/orange coursed brick with mortar lines
CONCRETE	Dark grey with form-board lines and aggregate detail
WOOD	Vertical timber panels with visible grain
MARBLE	Light stone with veining
W.TILE	White ceramic grid tiles

5.2 Floor Finishes

Six built-in floor finishes are available: Tile, Parquet, Marble, Concrete, Stone, and Carpet.

5.3 Application Scope

- ALL WALLS — applies the chosen finish to every wall simultaneously.
- SELECT — click individual walls in the viewport (orange outline appears), then apply the finish to selected walls only.

5.4 Additional Options

- Custom colour — a colour picker allows any solid wall colour independent of the texture presets.
- Tile size — a slider controls how large each texture tile appears (range: 0.3 m – 4 m).

6. API Reference

Base URL: <http://localhost:8000>

Endpoint	Method	Description
/health	GET	Health check — returns {"status": "ok"}
/upload	POST	Upload a floor plan file; returns job_id
/process/{job_id}	GET	Process the uploaded file; returns 3D model JSON

6.1 Process Endpoint Query Parameters

Parameter	Default	Description
scale	1.0	Unit scale factor (0 = auto-detect)
wall_height	3.0	Wall height in metres
wall_thickness	0.2	Fallback wall thickness in metres
pixels_per_meter	0	Must be 0 — triggers auto-detection from image dimensions

7. Known Limitations

The following issues have been identified during testing. They are documented transparently to guide future development.

Issue	Severity	Status
Window symbols only detected on vertical walls	Low	Open
Thin wall door detection (SCAN_HALF too wide)	Medium	Open
Phantom doors from tick marks near edges	Medium	Open
Wall thickness measurement accuracy	High	Open
Corner gap edges between perpendicular walls	High	Partial
VR headset integration (WebXR)	Future	Planned

8. Troubleshooting

8.1 API OFFLINE / Not Connecting

1. Ensure Uvicorn is running: `cd backend && uvicorn app.main:app --reload --port 8000`
2. Ensure you opened the viewer via HTTP (`http://localhost:3000`), not as a `file://` URL.
3. Open the browser console and run:
4.

```
fetch('http://localhost:8000/health').then(r=>r.json()).then(console.log).catch(console.error)
```

8.2 Walls Look Wrong or Missing

- Check the log panel in the viewer for PPM (pixels-per-meter) detection output.
- Upload the raw floor plan PNG, not a screenshot of the 3D viewer.

8.3 Textures Look Too Bright

- Switch to Blueprint mode and back to Realistic mode to re-apply defaults.
- Use the RESET ALL button and reapply your preferred texture.

8.4 Floor Plan Processes but Shows No Model

- Check the browser console for JavaScript errors.
- Verify the backend terminal logs for Python tracebacks.

8.5 Documentation Reference Files

File	Purpose
CHANGELOG.md	Full history of every change, organized by session
LEARNMAP.md	Hard-won lessons — what works, what broke, and why
PROJECTMAP.md	Complete map of every file, function, data flow, and open problem

— *End of Report* —